



▽ SESSION 08

Teaching **KelanaAI** to Know Its Users

Build KelanaAI with Python, Next.js & Amazon Bedrock — Session 8



Duration: 2 Hours



Week 3 · Authentication



Feature: User Authentication with JWT

Yesterday KelanaAI learned how to display trips.

Today KelanaAI learns who owns those trips.



SFIA Competency Card

Session 8 — Teaching KelanaAI to Know Its Users

FEATURE DELIVERED



User Authentication with JWT

PRIMARY SFIA SKILLS

PROG L2 Programming / Software Development

SCTY L2 Information Security

IDAM L2 Identity & Access Management

SWDN L2 Software Design

EVIDENCE — BY END OF SESSION, STUDENTS PRODUCE:

- ✓ User registration
- ✓ User login
- ✓ JWT authentication
- ✓ Protected API endpoints
- ✓ User-specific trip history
- ✓ Git commit

jwt · bcrypt · security Tech stack covered today

Learning Objectives

By the end of this session, students will be able to:

6 objectives · 2 hours

01



Explain authentication and authorization.

Who you are vs what you can access.

02



Register new users.

`POST /api/v1/auth/register`

03



Authenticate users using JWT.

JSON Web Tokens carry identity.

04



Protect FastAPI endpoints.

Require Bearer token on routes.

05



Associate trips with their owners.

`user_id` on every trip — backend-set.

06



Consume protected APIs from Next.js.

Send `Authorization` header on fetch.



Product Roadmap

Where we are — Week 3, Authentication

📍 You are here

WEEK 1 ✓	WEEK 2 ✓	WEEK 3 TODAY	WEEK 4 🔄
✓ Trip Summary	✓ PostgreSQL	✓ Trip Dashboard	✓ Conversation History
✓ Recommendation Engine	✓ Amazon Bedrock	▶ Authentication	✓ Deployment
✓ REST API	✓ Next.js Frontend	✓ RAG	✓ Capstone

→ Yesterday KelanaAI learned how to display trips. Today KelanaAI learns who owns those trips.



Story — Without Auth, Chaos

Imagine two users. Without authentication, they can see — and delete — each other's trips.

IMAGINE TWO USERS

😊 Alice → Japan Trip

😊 Bob → Korea Trip

WITHOUT AUTHENTICATION

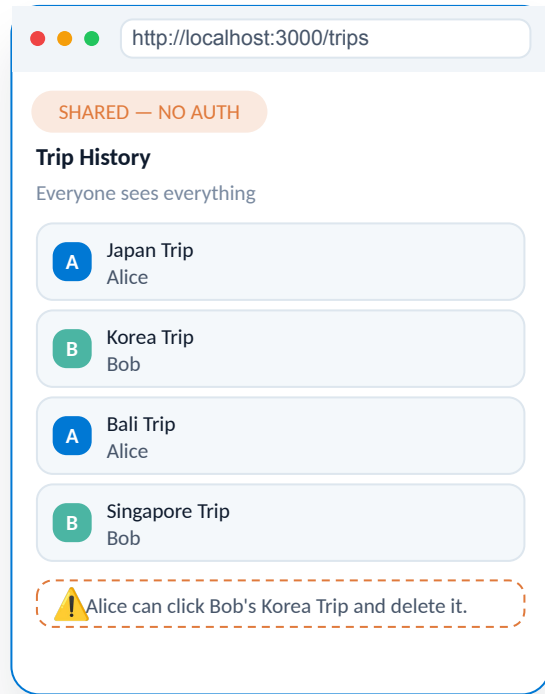
- ✗ Alice can see Bob's itinerary
- ✗ Bob can delete Alice's trip
- ✗ No privacy, no ownership

?

Would you trust an app like this?

Of course not.

→ Today we'll give KelanaAI its own identity system.



No users table. No user_id. No protection.



Technical Lesson

Eight parts — from auth concepts to protected multi-user APIs.

01

Auth vs Authz

02

DB Changes

03

Register

04

Login

05

Protect APIs

06

Ownership

07

Login Page

08

Logout

Never trust user input for ownership — the backend decides.



Part 1 — Authentication vs Authorization

Two related but different concepts. AuthN = who you are. AuthZ = what you can access.

AUTHENTICATION · AUTHN



Who are you?

Verifies identity. The user proves they are who they claim to be — usually with email + password.

EXAMPLES

- ✓ Login with email + password
- ✓ Google / GitHub OAuth login
- ✓ Biometric (fingerprint, face)

Happens once per session.

AUTHORIZATION · AUTHZ



What can you access?

Determines permissions. After identity is verified, the system checks what resources this user can read, modify, or delete.

EXAMPLES

- ✓ View only your own trips
- ✓ Edit your own profile
- ✓ Admin can see all users

Happens on every request.

✱ HOW THEY WORK TOGETHER



Login



Authentication



View only your own trips



Authorization

Part 2 — Database Changes

Add a users table. Add user_id to trips. Every trip now belongs to a user.

users (NEW TABLE)

PK	id	BIGINT
	name	VARCHAR(100)
	email	VARCHAR(255) UNIQUE
	password_hash	VARCHAR(255)

password_hash — never store plain text passwords.

trips (MODIFIED)

PK	id	BIGINT
FK	user_id	BIGINT (NEW)
	destination	VARCHAR
	days	INTEGER
	budget	FLOAT
	category	VARCHAR
	ai_recommendation	TEXT

user_id — links each trip to its owner.

RELATIONSHIP



One User → Many Trips

A user owns multiple trips. Each trip belongs to exactly one user. The user_id foreign key enforces this relationship.

WHY IT MATTERS

- ✓ Every query can filter by user_id
- ✓ Users see only their own trips
- ✓ Ownership is enforced at the DB level

MIGRATION

- ✓ Create users table
- ✓ ALTER TABLE trips ADD COLUMN user_id BIGINT NOT NULL;
- ✓ Update existing trips, add FOREIGN KEY, enforce NOT NULL

Existing trips get a default user_id or are deleted.

Part 3 — Register Endpoint

POST /api/v1/auth/register. Never store plain text passwords — hash them.

● ● ● POST /api/v1/auth/register — Request

REQUEST

```
{
  "name": "Alice",
  "email": "alice@email.com",
  "password": "password123"
}
```

● ● ● services/auth_service.py

```
import bcrypt

def hash_password(password: str) -> str:
    return bcrypt.hashpw(
        bytes(password, encoding="utf-8"),
        bcrypt.gensalt(),
    ).decode("utf-8")

def register(name, email, password):
    # NEVER store plain text - hash the password
    user = User(
        name=name,
        email=email,
        password_hash=hash_password(password)
    )
    db.add(user)
    db.commit()
    return user
```

SECURITY RULE



Never store plain text passwords

Hash passwords with bcrypt before saving. A hash is one-way — even if the database leaks, attackers cannot recover the original passwords.

WHY BCRYPT?

- ✓ One-way function (cannot be reversed)
- ✓ Salted automatically (defeats rainbow tables)
- ✓ Slow by design (defeats brute force)

VERIFICATION

- ✓ User enters password
- ✓ `pwd_context.verify()` compares it with the stored
- ✓ `login` succeeds only if they match

passlib + bcrypt — the Python standard.

Part 4 — Login Endpoint

POST /api/v1/auth/login. Verify credentials, return a JWT. The token represents the user's identity.

● ● ● POST /api/v1/auth/login — Response 200

RESPONSE

```
{
  "access_token": "eyJhbGciOiJIUzI...",
  "token_type": "Bearer..."
}
```

● ● ● services/auth_service.py

```
from jose import jwt

SECRET_KEY = "your-secret-key"

def login(email, password):
    user = db.query(User).filter(
        User.email == email
    ).first()

    # Verify password against stored hash

    if not pwd_context.verify(
        password, user.password_hash
    ):
        raise HTTPException(401)

    # Generate JWT
    token = jwt.encode(
        { "sub": user.id, "exp": ... },
        SECRET_KEY
    )
    return {"access_token": token}
```

JWT



The token = identity

A JSON Web Token (JWT) is a signed string that encodes the user's ID. The backend signs it with a secret key. The frontend stores it and sends it with every request.

JWT STRUCTURE

- Header — algorithm (HS256)
- Payload — {"sub": user_id, "exp": ...}
- Signature — HMAC of header+payload

TOKEN LIFECYCLE

- 1 User logs in
↓
- 2 Backend returns JWT
↓
- 3 Frontend stores token
↓
- 4 Token sent on every request

No server-side session — JWT is stateless.

Part 5 — Protect APIs

Previously anyone could access GET /api/v1/trips. Now only authenticated users can retrieve their trips.

PREVIOUSLY · OPEN



```
@app.get("/api/v1/trips")
def list_trip():
    return db.query(Trip).all()
```

Anyone could access it. No token required.



Every trip — every user's — is visible.

Alice sees Bob's trips. No privacy.

Limitation: no way to identify the caller.

NOW · PROTECTED



Require an Authorization header. Decode the JWT. Filter trips by the authenticated user.

```
# Require Authorization header

@app.get("/api/v1/trips")
def list_trip(user: User = Depends(get_current_user)):
    return db.query(Trip).filter(
        Trip.user_id == user.id
    ).all()
```

REQUEST HEADER

Authorization: Bearer eyJhbGciOiJI...

- ✓ Token required — 401 if missing
- ✓ Token verified — 401 if invalid
- ✓ Trips filtered by user.id

Alice sees only Alice's trips. Bob sees only Bob's.



Only authenticated users can retrieve their trips.

FastAPI Depends() injects the current user automatically.

Part 6 — Associate Trips with Users

When creating a trip, automatically assign `user_id` from the authenticated user — never from the frontend.

```
main.py

# The frontend NEVER sends user_id
# The backend sets it from the JWT

@app.post("/api/v1/trips")
def create_trip(
    request: TripRequest,
    user: User = Depends(get_current_user)
):
    trip = Trip(
        destination = request.destination,
        days         = request.days,
        budget       = request.budget,
        category     = get_trip_category(request.budget),
        daily_budget = calculate_daily_budget(
            request.budget,
            request.days
        ),
        # ownership - backend sets this
        user_id = user.id
    )

    db.add(trip)
    db.commit()
    return trip
```

SECURITY RULE



Never trust user input for ownership

The frontend no longer sends `user_id`. The backend extracts it from the JWT and sets it automatically. Users cannot create trips on behalf of other users.

WHAT CHANGED

- ✓ Frontend: removed `user_id` from request
- ✓ Backend: reads `user.id` from JWT
- ✓ DB: `trip.user_id` set automatically

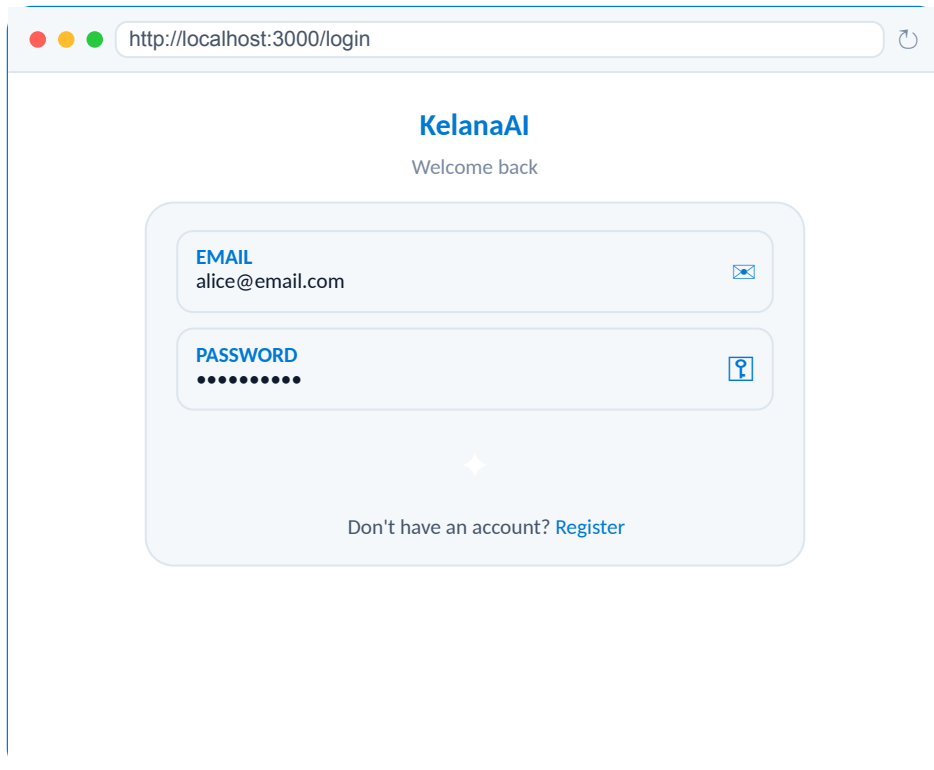
ATTACK PREVENTED

Without this, Alice could POST `{"user_id": 2}` and create a trip under Bob's account. The backend-set `user_id` closes this hole.

Ownership is a backend concern — never a frontend one.

Part 7 — Next.js Login Page

Build /login. Store the JWT. Redirect to dashboard. Every API call includes the Authorization header.



The screenshot shows a web browser window with the address bar displaying `http://localhost:3000/login`. The page content includes the KelanaAI logo and the text "Welcome back". Below this is a login form with two input fields: "EMAIL" with the value "alice@email.com" and a mail icon, and "PASSWORD" with masked characters and a key icon. A loading spinner is visible below the password field. At the bottom of the form, there is a link that says "Don't have an account? Register".

LOGIN WORKFLOW



Login → Token → Dashboard

After login, the frontend stores the JWT (localStorage or cookie) and redirects to /trips. Every subsequent API call includes the Authorization header.

THE FLOW

- 1 User enters email + password
- ↓
- 2 POST /api/v1/auth/login
- ↓
- 3 Receive JWT
- ↓
- 4 Store token (localStorage)
- ↓
- 5 Redirect to /trips

EVERY API CALL AFTER

No token = 401 Unauthorized.

Part 8 — Logout

Logging out means forgetting the user's identity on the client. Remove the token, redirect to /login.

LOGOUT WORKFLOW



User clicks Logout button



Remove JWT from localStorage



Clear user state from React



Redirect to /login

CODE

```
localStorage.removeItem( "token" )  
router.push( "\\login" )
```

No API call needed — JWT is stateless.

CONCEPT



Logout = forget the token

Because JWT is stateless, the backend doesn't track sessions. Logout happens entirely on the client — remove the stored token and redirect.

WHAT HAPPENS

- ✓ Token removed from localStorage
- ✓ React state cleared (no current user)
- ✓ User redirected to /login

EDGE CASE

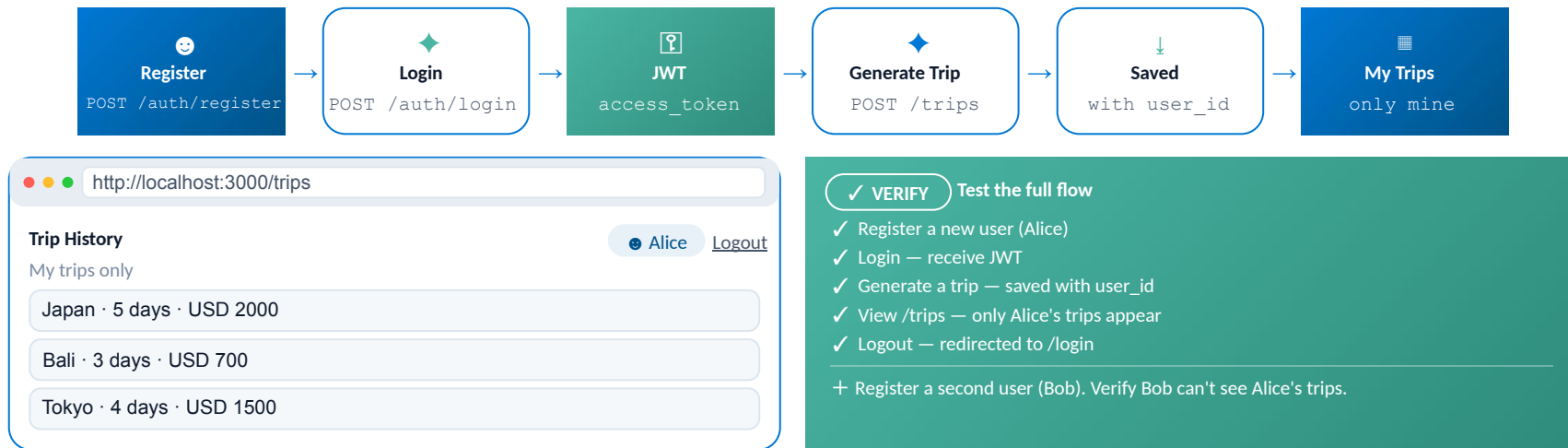
Existing tokens remain valid until they expire. For instant logout, use a token blacklist — but that requires server-side state.

JWT EXPIRY

Short-lived tokens are more secure.

Hands-on Lab — Full Auth Flow

Register → Login → JWT → Generate Trip → Saved with user_id → View My Trips



Build the full auth flow. Register 2 users. Verify they can't see each other's trips.

Time-box: 35 min

Challenge

Build a profile page. Personalize the navigation bar.

◆ Solo Challenge

CORE CHALLENGE — PROFILE PAGE



Create /profile

A personal page showing the current user's info and trip count.

DISPLAY:

- ✓ Name
- ✓ Email
- ✓ Total Trips Generated

ENDPOINT HINT:

```
GET /api/v1/auth/me # Returns current user from JWT
```

◆ Use the JWT to identify the user — no user_id in the URL.

BONUS



Personalized Welcome

Show a welcome message in the navigation bar after login.

DISPLAY IN NAVBAR:

```
Welcome back,  
Alice 🖐️
```

IMPLEMENTATION

Store user name in React state
Render in navbar after login
Clear on logout

♥ Small touch, big UX win.

Git Checkpoint

Commit today's work. KelanaAI learned how to recognize its users.

```
bash -  
kelana=ai  
  
# stage today's auth work  
git add .  
  
# commit with a descriptive message  
git commit -m "Add JWT authentication"  
  
# push to GitHub  
git push
```



Today KelanaAI learned how to recognize its users.

☒ Tag your commit: `git tag session-8`

Mini Quiz

Quick check — discuss with your neighbour, then we'll review.

◆ 4 Questions

Q1



What is the difference between authentication and authorization?

Hint: Who you are vs what you can access.

Q2



Why should passwords be hashed?

Hint: Think about database leaks.

Q3



Why shouldn't the frontend send user_id?

Hint: Who decides ownership?

Q4



What is stored inside a JWT?

Hint: It's a signed string with user info.



Discuss for 5 minutes. We'll review answers together.



Pair up

Homework & What's Next

Lock down all CRUD endpoints tonight — and tomorrow, KelanaAI learns to read external knowledge.

→ Wrap-up

Homework — Protect All APIs



Ensure every CRUD endpoint respects ownership.

- 01** ☒ View: only own trips
GET /trips filters by user_id — already done today.
- 02** ☐ Update: reject other users' trips
PUT /trips/{id} — return 403 if user_id mismatches.
- 03** ☐ Delete: reject other users' trips
DELETE /trips/{id} — return 403 if user_id mismatches.

↑ Commit and push your solution.

NEXT SESSION — 9



KelanaAI Learns to Read

Today KelanaAI learned who its users are. Each traveler has a secure, private workspace. However...

"KelanaAI still relies only on the knowledge inside the Large Language Model."

Tomorrow: RAG with Amazon Bedrock Knowledge Bases

- Upload travel guides, visa requirements, company docs
- KelanaAI answers using YOUR trusted data
- No more hallucinations on factual questions

KelanaAI moves beyond general knowledge — it becomes an expert on your data.



◆ MILESTONE ACHIEVED

KelanaAI became a **secure multi-user** application

Instead of being a shared prototype, it now supports individual user accounts, protected APIs, and private travel histories.



Authentication

Identifies the user via JWT.



Authorization

Protects user data per request.



Backend Owns Access

Ownership never trusted to frontend.



Frontend Presents

UI sends token, displays data.



KelanaAI is ready to move beyond general knowledge — tomorrow it learns to read your trusted data with RAG.

Authentication + Authorization + Ownership = production-ready security.